

Présentation du Frontend Angular et des appels REST dans TaskManager Microservices

1. Introduction

- Présentation du projet TaskManager Microservices

Connexion

Nom d'utilisateur

Mot de passe

Se connecter

Mes tâches

Deconnexion

Ajouter

task1

Description 1

Statut : En cours

Terminer Supprimer

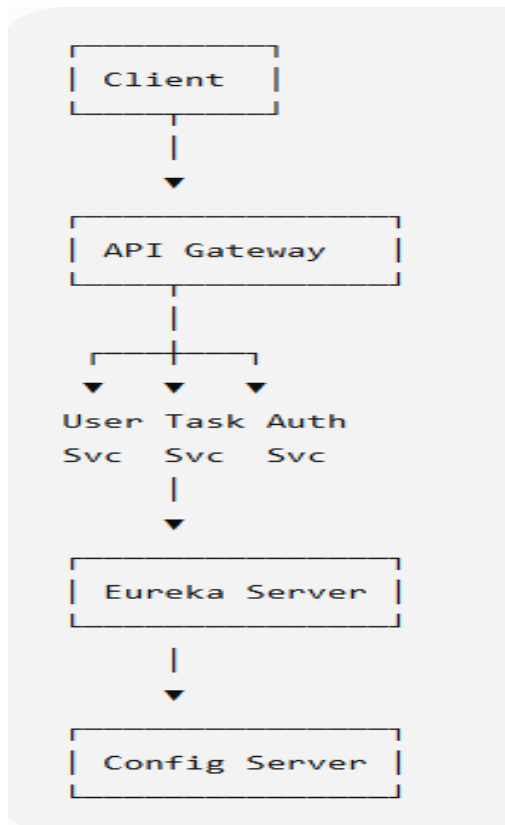
task 2

Description 2

Statut : En cours

Terminer Supprimer

- Architecture générale :



- Rôle du frontend Angular

2. Architecture du frontend

Présentation de l'organisation du projet Angular :

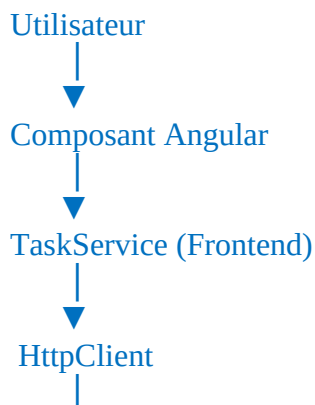
frontend-angular

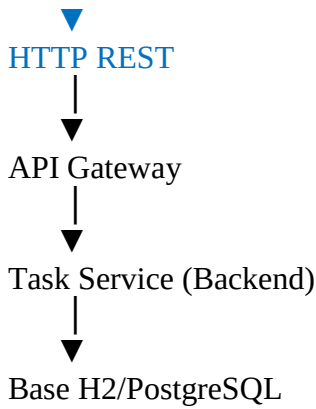


Explication du rôle de chaque dossier.

3. Communication Angular ↔ Spring Boot

Schéma :





Ce qui correspond au cycle complet d'une requête HTTP.

4. Le service Angular

Principe de séparation des responsabilités :

- le composant gère l'interface utilisateur;
- le service appelle les API Java ;
- HttpClient réalise les requêtes HTTP.

Exemple d'appel :

```
getTasks(): Observable<Task[]> {  
    return this.http.get<Task[]>(this.apiUrl);  
}
```

appel l'API get() au travers de la requête http à laquelle, l'on passe l'url de l'API.

5. Les Observables

Définition

Un **Observable** représente un flux de données pouvant émettre plusieurs valeurs au cours du temps.

Angular utilise les Observables pour :

- GET
- POST
- PUT
- DELETE
- événements utilisateur
- WebSocket

Fonctionnement

Observable () ==> géré au niveau du service task-service

↓
subscribe() ==> géré au niveau du composant métier tasks.ts

↓
next()
error()
complete()

Exemple :

```
this.taskService.getTasks()  
    .subscribe(tasks => {  
        this.tasks = tasks;  
    });
```

inscription au flux de gestion des taches, taskService.

6. Les Promises

Une **Promise** représente une seule réponse asynchrone.

Exemple :

```
async loadTasks() {  
  const tasks = await firstValueFrom(  
    this.taskService.getTasks()  
  );  
}
```

7. Observable vs Promise

Observable

Plusieurs valeurs

Annulable

Lazy

Très utilisé dans Angular

Promise

Une seule valeur

Non

Exécution immédiate

Utilisation ponctuelle

8. Introduction à RxJS

RxJS : bibliothèque permettant de programmer à l'aide d'Observables et de manière réactive.

Principaux opérateurs :

- map
- tap
- filter
- switchMap
- catchError
- retry
- finalize

Exemple :

```
return this.http.get<Task[]>(url)  
  .pipe(  
    tap(() => console.log("Chargement")),  
    catchError(this.handleError)  
  );
```

9. Cycle complet d'un appel API

Utilisateur



Clique sur "Ajouter"

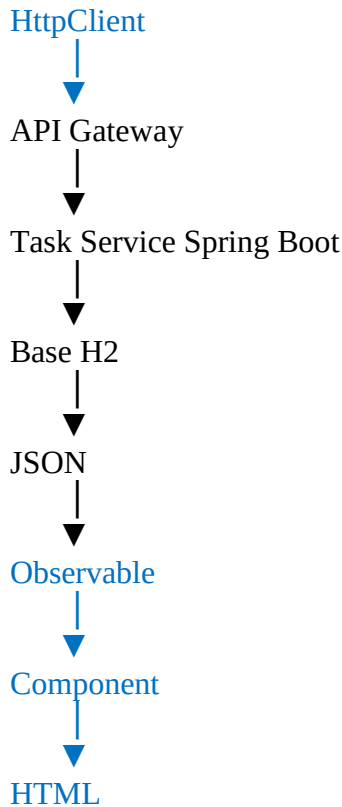


Component



TaskService





10. Coimposants essentiels du frontend TaskManager

- auth.service.ts : pour l'authentification
- tasks.ts : pour la gestion des tâches en utilisant task.service.ts

11. Bonnes pratiques

- Centraliser tous les appels REST dans les services.
- Utiliser les Observables avec pipe().
- Gérer les erreurs avec catchError().
- Utiliser un Interceptor pour le token JWT.
- Éviter les abonnements inutiles.

11. Conclusion

Le frontend Angular du projet TaskManager s'appuie sur **HttpClient**, **RxJS** et les **Observables** pour communiquer efficacement avec les microservices Java. Cette architecture favorise une séparation claire des responsabilités, une gestion réactive des données et une meilleure maintenabilité. Les Promises restent utiles dans certains cas, mais les Observables sont le mécanisme recommandé par Angular pour les appels HTTP et les flux asynchrones.